

Seguridad implementada en PPC Manager

Mapa de los requisitos de seguridad definidos en los documentos del curso — modelo de negocio, ciber-resiliencia y sistema de autenticación — contra lo que quedó realmente implementado en la aplicación (Angular + NestJS + Supabase), organizado por la tríada CIA, con la guía paso a paso de cómo usar cada control.

Autores: Anthony Fabian Ramirez Orellana · Nery Otoniel Colorado Menchu

Sección: A

De los documentos del curso a la aplicación

Cada control implementado responde a un requisito concreto de alguno de los cuatro documentos entregados. Esta tabla es el hilo conductor del resto del dossier.

DOCUMENTO FUENTE	REQUISITO	ESTADO	DÓNDE VIVE
Tarea S8 — Sistema de Autenticación	TOTP estándar para todos los usuarios	Implementado	supabase.auth.mfa
Tarea S8 — Sistema de Autenticación	Segundo factor reforzado obligatorio para el rol administrador	Adaptado	mfaEnforcementGuard
Tarea S8 — Sistema de Autenticación	Códigos de recuperación de un solo uso	Implementado	mfa_recovery_codes
Tarea #7 — CiberResiliencia	RBAC por rol de usuario	Implementado	profiles + RolesGuard
Tarea #7 — CiberResiliencia	Logs de auditoría por acción crítica	Implementado	audit_log
Tarea #7 — CiberResiliencia	SBOM (CycloneDX) + escáner de dependencias en CI/CD	Implementado	.github/workflows/security.
Modelo de negocio y seguridad	Validación/sanitización de cargas Excel/CSV anti-inyección	Implementado	common/excel.util.ts
Modelo de negocio y seguridad	Cifrado en tránsito (TLS) y en reposo (AES-256)	Delegado a infraestructura	Lo provee Supabase/el hosting, no e código de la app
Modelo de negocio y seguridad	ETL, multi-tenant por vendedor, OAuth Amazon Seller / Helium 10 / Groq	Fuera de alcance	Pertenece a la arquitectura teórica c curso, no a esta app

Roles y control de acceso (RBAC + RLS)

Solo quien debe ver o modificar algo puede hacerlo. Base de todo lo demás: MFA, auditoría y validación se apoyan en saber quién es quién.

Administrador vs. Usuario

Origen: Tarea #7 — CiberResiliencia · «RBAC por rol de usuario»

Tabla `profiles` vinculada 1:1 a `auth.users` vía trigger, con un campo `role` (`admin` / `user`). Row-Level Security con una función `is_admin()` `security definer` evita la recursión de policies. En el backend, `RolesGuard` lee el rol desde el token y lo compara contra `@Roles('admin')` en cada endpoint protegido. Un admin no puede degradar su propia cuenta.

- 1
- Entra a `/admin` en el sidebar (solo visible si tu rol es admin).
- 2
- Verás la tabla de usuarios con su correo y su rol actual.
- 3
- Haz clic en el badge de rol de **otro** usuario (tu propia fila queda fija, sin dropdown).
- 4
- Elige **Administrador**

Usuario

Administrador — el cambio se aplica al instante y queda registrado en la bitácora.

Verificación en dos pasos (TOTP)

El código de un solo uso vive en el dispositivo, no depende de SMS ni de una red de telefonía vulnerable a SIM swapping.

2FA con Google Authenticator / Authy

Origen: **Tarea S8** · «TOTP estándar para todos, reforzado obligatorio para admin»

Usa la API nativa `supabase.auth.mfa` (enroll → challenge → verify), sin dependencias extra. El rol `admin` no puede navegar la app sin un factor `verified : mfaEnforcementGuard` lo intercepta y lo manda a configurarlo. El resto de los usuarios lo activa cuando quiere, desde el sidebar.

- 1 En el sidebar, entra a

Seguridad (2FA)

- 2 Clic en

Generar código QR

- 3 Escanéalo con Google Authenticator, Authy o Microsoft Authenticator (o pega la clave manual que aparece debajo).
- 4 Escribe el código de 6 dígitos que te da la app y confirma.
- 5 En tu próximo login, después de la contraseña, el sistema te pedirá ese código de 6 dígitos en

/mfa-
verify

Códigos de recuperación

Supabase Auth no trae códigos de respaldo de fábrica — se construyeron a mano para cubrir exactamente el escenario que pedía el documento: perder el dispositivo autenticador.

10 códigos de un solo uso, con hash bcrypt

Origen: **Tarea S8** · «se recomienda la generación de códigos de recuperación estáticos de un solo uso»

Tabla `mfa_recovery_codes` (solo accesible por el backend, sin políticas de RLS para nadie más). Los códigos se guardan como hash, nunca en texto plano. Al consumir uno válido, una función `security definer ( admin_delete_all_mfa_factors )` borra el factor TOTP del usuario directamente en `auth.mfa_factors`, obligándolo a re-inscribirse.

1 Con el 2FA ya activo, ve a

**Seguridad
(2FA)**

**Generar
nuevos
códigos
de
recuperación**

2 Aparecen 10 códigos (formato

XXXXX)— cópialos o descárgalos como
XXXXX

.txt .

**No
se
vuelven
a
mostrar.**

3 Si pierdes tu celular: en

/mfa-, clic en
verify

**«¿Perdiste
tu
dispositivo?**

**Usa
un
código**

de
recuperación»

- 4 Pega uno de los códigos guardados — cada uno funciona una sola vez.
- 5 Al usarlo, tu 2FA queda desactivado y (si eres admin) el sistema te exige configurar uno nuevo de inmediato.

CONFIDENCIALIDAD

Política de contraseñas

La contraseña sigue siendo el primer factor — que sea débil anula todo lo demás.

Mínimo 8 caracteres, mayúscula, minúscula y número

Origen: **Modelo de negocio y seguridad** · «hash bcrypt/Argon2, contraseña robusta»

El hash de contraseñas lo hace Supabase Auth internamente (nunca tocamos texto plano). Del lado de la app se agregó un validador de complejidad en el formulario de registro y se subió el mínimo también en el propio proyecto de Supabase.

Frontend `strongPasswordValidator()` — 8+ caracteres, mayúscula, minúscula, número.

Supabase Authentication → Providers → Email → Minimum password length = 8.

Pendiente «Leaked Password Protection» (contraseñas filtradas) existe en Supabase pero es exclusivo del plan Pro — no disponible en este proyecto.

Cierre de sesión por inactividad

Una sesión abierta y sin vigilancia es una puerta abierta, sobre todo en un equipo compartido.

15 minutos sin actividad → logout automático

Origen: extensión propia sobre la tríada CIA (confidencialidad de sesión)

`IdleService` escucha mouse, teclado, scroll y touch mientras el shell autenticado está activo. Sin interacción durante 15 minutos, cierra la sesión de Supabase y redirige a `/login` con un aviso.

- 1 Inicia sesión normalmente — el temporizador arranca solo, sin configuración manual.
- 2 Deja la pestaña abierta sin tocar nada por 15 minutos.
- 3 La sesión se cierra sola y ves: «Tu sesión se cerró por inactividad. Vuelve a iniciar sesión.»

Bitácora de auditoría inmutable

Un registro que nadie revisa no protege nada — pero primero tiene que existir, y no se puede alterar después del hecho.

Quién, qué, cuándo y desde dónde

Origen: **Tarea #7 — CiberResiliencia** · «logs de auditoría inmutables por cada acción crítica»

Tabla `audit_log` con `user_id` , `action` , `entity_type` / `entity_id` , `ip_address` y `created_at` . RLS solo permite `SELECT` a administradores — no existe policy de `UPDATE` ni `DELETE` para nadie, ni siquiera vía API con la service role.

<code>login</code>	cada inicio de sesión exitoso, con IP.
<code>login_failed</code>	intentos fallidos, incluso de correos que no existen (endpoint público, con throttle propio de 10/min).
<code>role_change</code>	quién cambió el rol de quién, y a qué rol.
<code>mfa_recovery_used</code>	uso de un código de recuperación (éxito o fallo).

1 Como admin, entra a

/admin

2 Baja hasta

«Bitácora de auditoría»

3 Cada fila muestra usuario, acción, detalle, IP y fecha — sin necesidad de tocar la base de datos directamente.

Validación de entradas en toda la API

Que un dato mal formado se rechace en la puerta, no que se descubra tres pasos después.

ValidationPipe global + DTOs

Origen: extensión propia (integridad de datos, OWASP ASVS — validación de entrada)

NestJS valida y transforma cada `@Body()` contra una clase con `class-validator`.
`whitelist` elimina campos no declarados y `forbidNonWhitelisted` rechaza la petición entera si trae algo extra.

```
# Probado en vivo contra /audit/failed-login
# sin correo
curl -X POST .../audit/failed-login -d '{}'
```

→ 400 {"message":["email must be an email"]}

```
# campo no declarado en el DTO
curl -X POST .../audit/failed-login -d '{"email":"a@b.com","hack":"1"}'
```

→ 400 {"message":["property hack should not exist"]}

INTEGRIDAD

Anti-inyección de fórmulas en Excel/CSV

Un archivo que un usuario del equipo sube hoy puede volver a abrirse en Excel mañana —ahí es donde ataca la inyección de fórmulas.

OWASP CSV/Formula Injection

Origen: **Modelo de negocio y seguridad** · «validación y sanitización de archivos Excel/CSV cargados»

Cualquier valor de celda que empiece con `=`, `+`, `-`, `@`, tabulador o retorno de carro se antepone con una comilla simple al parsear el archivo (en `listings`, `search-terms` y `ppc`), neutralizándolo como texto literal sin tocar los datos legítimos que solo tienen un guion en medio del texto.

Entrada en el Excel:	<code>=cmd '/c calc'!A1</code>
Guardado en la base:	<code>'=cmd '/c calc'!A1</code> ← inofensivo si se reabre en Excel
Entrada en el Excel:	Producto normal con - guion
Guardado en la base:	Producto normal con - guion ← intacto

DISPONIBILIDAD

Rate limiting y cabeceras de seguridad

Que el servicio siga respondiendo aunque alguien lo golpee con tráfico automatizado.

@nestjs/throttler + helmet

Origen: extensión propia sobre la tríada (disponibilidad, mitigación de fuerza bruta)

Global	100 solicitudes / minuto por IP en toda la API.
<code>/audit/failed-login</code>	límite propio de 10/min — evita usarlo para inundar la bitácora.
Cabeceras	<code>helmet()</code> agrega <code>X-Content-Type-Options</code> , <code>X-Frame-Options</code> y el resto del set estándar de OWASP en cada respuesta.

SBOM, escaneo de dependencias y CI

El código propio puede estar limpio y aun así heredar una vulnerabilidad de un paquete de npm de tercer nivel.

CycloneDX + npm audit en cada push

Origen: **Tarea #7 — CiberResiliencia** · «generar SBOM en cada CI/CD, bloquear versiones con vulnerabilidades críticas»

Ambos proyectos (`frontend/` y `backend/`) generan un inventario de dependencias en formato CycloneDX y corren un audit que falla el build si aparece algo de severidad alta o crítica. Se eliminó `@nestjs/mau` (dependencia de despliegue sin uso real) porque era la fuente de las únicas vulnerabilidades altas del proyecto.

```
npm run sbom          # genera sbom.json (CycloneDX 1.6)
npm run security:audit # npm audit --audit-level=high
```

1 Cada

push o

pull dispara
request

`.github/workflows/security.yml`

2 Corre el audit de dependencias en

`frontend/`

`backend/` separado.

3 Si hay algo alto/crítico, el workflow falla y bloquea el merge.

4 Si pasa, sube el

`sbom.json` de cada proyecto como artifact descargable del run.

Qué quedó deliberadamente fuera

Los documentos del curso describen una arquitectura teórica más amplia (multi-tenant, React/Next.js, MongoDB, integraciones con Amazon Seller Central, Helium 10 y Groq). Esta app real no la usa, así que estos puntos no aplican o dependen de infraestructura que no se controla desde el código.

Arquitectura teórica distinta

ETL hacia MongoDB, multi-tenant por vendedor, OAuth con Amazon Seller Central (LWA), Helium 10, Google Sheets, Groq/IA. No existen en esta app.

Delegado al proveedor cloud

TLS 1.2/1.3, cifrado AES-256 en reposo, firewalls, IP allowlist/VPN de los paneles de Supabase y Netlify, redundancia multi-zona.

Proceso, no código

Revisión periódica del log de auditoría, procedimiento de respuesta a incidentes, verificación cruzada fuera de banda para cambios de pago.

Limitado por el plan

«Leaked Password Protection» de Supabase existe pero es exclusiva del plan Pro — documentado como aceptado, no implementado.

Fuentes: Tarea S8 — Sistema de Autenticación · Tarea #7 — CiberResiliencia · Modelo de negocio y solución de seguridad informática · Amazon_SEO_Optimizer_Seguridad.pdf